



Riassunto Visivo: Design by Contract (DbC)

Il Design by Contract (Progettazione per Contratti) trasforma le interfacce software da semplici "firme" di metodi a veri e propri **contratti legali e matematici** tra moduli software.

1. I Fondamenti: Correttezza e Triple di Hoare

La correttezza di un software non è un valore assoluto. L'istruzione $x = x + 1$ non è giusta o sbagliata di per sé, ma lo è *solo rispetto alla sua specifica*.

Per formalizzarlo si usano le **Triple di Hoare**:

$\{ P \} A \{ Q \}$

- **{ P } = Precondizione:** Lo stato *prima* dell'esecuzione.
- **A = L'operazione:** Il codice eseguito.
- **{ Q } = Postcondizione:** Lo stato *garantito dopo* l'esecuzione.

(Semantica: Se parto da uno stato in cui P è vera ed eseguo A , il programma terminerà in uno stato in cui Q è vera).

Il Paradosso del Programmatore (Domanda da Lode)

Dal punto di vista di chi scrive il codice dell'operazione **A** (il Fornitore):

- La **migliore precondizione è false**: Più la precondizione è stringente (forte), meno casi devo gestire. Se chiedo false, l'utente non potrà mai invocare il metodo, quindi io non devo fare nulla!
- La **migliore postcondizione è true**: Più è debole, meno garanzie devo dare. Qualsiasi cosa faccia il mio programma, andrà bene.

2. La Matrice del Contratto (Diritti e Doveri)

Come in un contratto reale, ci sono due attori: il **Cliente** (chi chiama il metodo) e il **Fornitore** (chi implementa il metodo).

Attore	● OBBLIGHI (Doveri)	● BENEFICI (Diritti)
Cliente	Deve garantire che la Precondizione sia vera prima di chiamare il metodo.	Ha la garanzia di ottenere il risultato descritto dalla Postcondizione .

Fornitore	Deve svolgere il lavoro e garantire che la Postcondizione sia vera alla fine.	Può dare per scontato che la Precondizione sia vera (Non deve scrivere codice per controllarla!).
------------------	--	--

 **Regola Aurea:** Se il Cliente viola la Precondizione (es. passa un array vuoto a una funzione che chiede un array pieno), il Fornitore è **sollevato da ogni obbligo**. Può crashare, andare in loop o formattare il disco: *la colpa è del Cliente*.

3. Le Tre Asserzioni (L'Anima del DbC)

Come si traduce un Modello Matematico (ADT) in codice Java tramite il DbC? Usando 3 tipi di asserzioni:

1. **Precondizioni (Require):** I vincoli di input.
2. **Postcondizioni (Ensure):** Le garanzie di output. Spesso si usa la parola chiave `old` per confrontare lo stato finale con quello iniziale (es. `size == old.size + 1`).
3. **Invarianti di Classe (Invariant):** Proprietà di base che definiscono la validità dell'oggetto (es. in un Conto Bancario, `saldo >= 0`).

Gli Stati Stabili

Quando deve essere vero l'Invariante di classe? Non sempre! Durante l'esecuzione di un metodo, l'oggetto può essere temporaneamente "rotto". L'invariante deve valere solo negli **Stati Stabili**:

- Subito dopo la creazione (costruttore).
- *Prima* che inizi l'esecuzione di un metodo pubblico.
- *Subito dopo* la fine di un metodo pubblico.

4. Regole Formali di Correttezza (L'Induzione)

La correttezza di una classe si dimostra matematicamente come una **Dimostrazione per Induzione**:

- **Regola C1 (Passo Base - Creazione):** {DefaultC and pre_p} Costruttore {post_p and INV}
Spiegazione: Il costruttore prende le variabili vuote, accetta argomenti validi e genera un oggetto che rispetta l'Invariante globale.
- **Regola C2 (Passo Induttivo - Metodi):** {pre_r and INV} Metodo_Pubblico {post_r and INV}
Spiegazione: Se il metodo parte da una situazione stabile (INV) con dati validi (pre), alla fine del suo lavoro l'Invariante globale sarà ancora salvo.

5. Ereditarietà e "Subappalto Onesto" (Liskov)

Cosa succede alle asserzioni quando creo una Sottoclasse? Qui entra in gioco il **Principio di**

Sostituzione di Liskov (LSP), noto nel DbC come regola del "Subappalto".

Se la Classe Base appalta il lavoro alla Sottoclasse, la Sottoclasse deve essere un fornitore *onesto*:

- **Precondizioni:** Possono solo essere **INDEBOLITE** (Meno esigenti). La sottoclasse non può pretendere dal cliente più di quanto chiedesse il padre.
- **Postcondizioni:** Possono solo essere **RAFFORZATE** (Più generose). La sottoclasse deve garantire *almeno* quello che garantiva il padre, o persino di più.
- **Invarianti:** Vengono ereditati e sommati (Invariante Cumulativo).

(Se non rispetti queste regole, il polimorfismo si rompe e il programma va in crash).

RIPASSO DOMANDE DA ESAME (Formula "30 e Lode")

Domanda 1: "Nel Design by Contract, di chi è la responsabilità di validare l'input di un metodo?"

- *Risposta da Lode:* "La responsabilità ricade esclusivamente sul **Cliente** (chi chiama il metodo), che ha l'obbligo di soddisfare la Precondizione. Il Fornitore (chi esegue il metodo) ha il diritto di assumere che la Precondizione sia vera e non deve scrivere alcun blocco if per validare i dati di input. (Attenzione: i dati utente o di rete esterni vanno filtrati *prima* da moduli di validazione appositi, non dalle asserzioni di contratto software-to-software)."

Domanda 2: "Cosa afferma la Regola del Subappalto (legata al Principio di Liskov) riguardo a Precondizioni e Postcondizioni quando si eredita una classe?"

- *Risposta da Lode:* "Afferma che una sottoclasse deve essere un fornitore onesto e non può sorprendere negativamente il cliente che usa il polimorfismo. Pertanto, la sottoclasse può solo **indebolire le precondizioni** (chiedere di meno al cliente) e **rafforzare le postcondizioni** (garantire di più al cliente). Gli invarianti, invece, si sommano cumulativamente."

Domanda 3: "Quando deve essere garantita la validità dell'Invariante di una Classe?"

- *Risposta da Lode:* "L'Invariante non deve essere vero in ogni singolo istante di vita del software, poiché durante l'esecuzione sequenziale di un metodo lo stato interno può essere momentaneamente incoerente. L'Invariante deve valere unicamente negli **Stati Stabili**: al termine dell'esecuzione dei costruttori e ai 'confini' dei metodi pubblici (esattamente prima dell'invocazione e subito dopo il ritorno)."

Domanda 4: "Mi spieghi perché, dal punto di vista puramente teorico del fornitore, false è la migliore precondizione possibile?"

- *Risposta da Lode:* "In logica booleana (e nelle Triple di Hoare), un'implicazione il cui antecedente è falso risulta sempre vera. Se il fornitore imposta false come precondizione, significa che non esiste alcuno stato di input valido per chiamare la sua funzione. Di

conseguenza, il fornitore non dovrà mai fare alcuno sforzo computazionale e il suo contratto non potrà mai essere violato da parte sua."

Curiosità/Bonus per l'orale (Il crash di Ariane 5): Se il prof ti chiede a cosa serve il DbC nella realtà, cita il razzo Ariane 5. Esplose nel 1996 a causa di un *overflow* nella conversione di un numero. Il modulo fu riutilizzato dal vecchio Ariane 4 senza test. Se l'ingegnere avesse scritto una **Precondizione formale** del tipo Require: $velocita_orizzontale < X$, i sistemi di testing avrebbero subito segnalato che le velocità del nuovo razzo violavano il contratto di quel vecchio modulo, salvando 500 milioni di dollari!